Robust Imitation Learning for Mobile Robots

William Tremaglio
Department of Computer Science
University College London
London, United Kingdom

***Abstract*—We clone an expert controller for a simulated mobile manipulator: a four-wheeled base drives toward a target and parks, then a 7-DoF arm reaches it, and an episode succeeds if the hand ends within $1\,\mathrm{cm}$. Plain cloning of the supplied expert failed in closed loop, and diagnosing why shaped our method. We redesigned the expert so that its commands depend only on measurable state, gave the network only the quantities the expert's control laws read, and gated the base and arm commands with a separate parked head. DART noise, prioritized replay, and an ensemble of three networks then brought a 6,801-weight multilayer perceptron to expert level: on 100 held-out final episodes it reached every target at $1\,\mathrm{cm}$ and 99 at $5\,\mathrm{mm}$, against the expert's 100 and 100, deciding in $38\,\mu\mathrm{s}$. Every choice was made in closed loop, on seed sets kept apart for choosing, confirming, and reporting. A 10-step history, an LSTM, and gradient-boosted trees matched this accuracy at up to 340 times the decision time. The policy withstands command noise, but strong sensor noise, wider task ranges, and sideways pushes expose its limits; adding 300 pushed demonstrations raised success under $0.1\,\mathrm{m/s}$ pushes from 66 to 97 at a small nominal cost. A diagnosed negative result shows why DAgger fails when the expert's arm keeps hidden state.**

***Index Terms*—behavior cloning, imitation learning, mobile manipulation, DART, prioritized replay, causal confusion, robustness**

I. Introduction

A. Task

We were given a PyBullet simulation of a 7-DoF Kinova arm on a four-wheeled Robotnik base, with dynamics from Pinocchio, and a scripted expert that drives the base toward a target, stops at a standoff distance, and moves the hand onto the target (Fig. 1). The task is to train a model that clones this behavior from demonstrations and generalizes to new targets and starting poses.

B. Constraints and Assumptions

The simulation steps at $1\,\mathrm{kHz}$, and our policy decides at $100\,\mathrm{Hz}$. Each target lies in a random direction, 0.6 to $1.6\,\mathrm{m}$ from the origin horizontally and 0.3 to $1.0\,\mathrm{m}$ high; the base starts at rest within $0.5\,\mathrm{m}$ of the origin at any bearing, with the arm in its default pose and the fingers held fixed. The arm joints have fixed limits, and the wheels meet a flat floor through stiff contact with friction and damping, under standard gravity. An episode succeeds if the hand ends within $1\,\mathrm{cm}$ of the target; it cannot end before $8\,\mathrm{s}$ and is capped at $40\,\mathrm{s}$. We assume that the robot, the simulation, and the success test are fixed, but that the expert may be redesigned. Training assumes perfectly measured state; Section IV-E tests what happens without it.

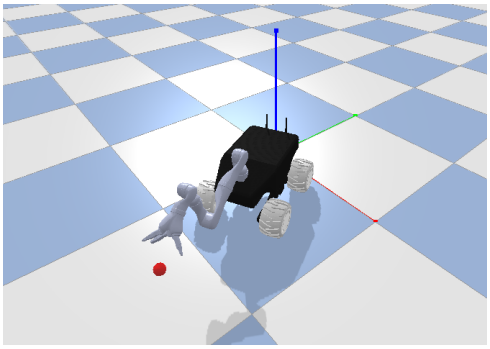


Fig. 1. The robot in the PyBullet simulation, reaching for the target (red sphere).

C. Contributions

We make the expert memoryless (Section II-A), justify every input, output, and network size in closed loop (Sections II-C and II-D), test the final policy outside its training conditions, and fix its largest weakness by training on pushed demonstrations (Section IV-E). We also diagnose why DAgger fails here (Section IV-F).

II. Methodology

A. Expert Redesign

The template expert works in phases: the base turns on the spot toward a goal pose, drives to it, and turns to face the target, and the arm then reaches through damped differential inverse kinematics and a joint PD controller with gravity compensation. It is hard to clone for two reasons. Its commands depend on hidden state, the current phase, so the same observed state can call for a straight drive or a turn on the spot. And its goal lies on the line from the world origin to the target, so the robot cannot infer where to stop from where the target is relative to itself. On 100 held-out episodes, it reached 95 targets, and 99 with a rule that realigns the base when it drifts off course.

We therefore made the base controller memoryless, keeping the template's arm. At every step, the base computes its heading error to the target and its signed distance from the *standoff circle* of radius $s(z)$ around the target, where z is the target's height. The turning rate is proportional to the heading error (gain 3, at most $1\,\mathrm{rad/s}$). The forward speed is proportional to the distance error (gain 1, between 0.05 and $0.25\,\mathrm{m/s}$), negative inside the circle, and scaled down linearly

to zero at a heading error of $0.5\,\mathrm{rad}$, so a base facing away first turns on the spot, while one roughly facing the target turns and drives at once. Within $3\,\mathrm{cm}$ of the circle and $0.05\,\mathrm{rad}$ of facing the target, or after a $25\,\mathrm{s}$ timeout, it sets a *parked* latch, its only memory, and the arm reaches. Because the goal is defined relative to the target, the network can observe it (Section II-C).

The standoff table, tuned one point at a time on the development seeds and confirmed on the confirmation seeds (Section III), is $s = 0.80$, 1.00, 0.75, and $0.35\,\mathrm{m}$ at heights of 0.1, 0.5, 1.1, and $1.25\,\mathrm{m}$. Against the realigned template, the redesigned expert reaches 100 targets instead of 99, cuts the median final error from 7.2 to $1.8\,\mathrm{mm}$, and parks in $3.8\,\mathrm{s}$ instead of $5.4\,\mathrm{s}$. It is also far easier to clone: the same network and recipe reached 65 targets on the realigned template's demonstrations and 96 on the redesigned expert's.

B. Data Collection

Each episode's target and start are drawn from its seed within the task's ranges, so a seed always reproduces its episode. Episodes run in parallel processes, each resetting its simulation instead of rebuilding it, which collects 600 in about two minutes. At every policy decision, we record the raw state, the expert's commands, and its parked flag; we store the rows as NumPy `.npz` files, which keep their types and load quickly, and compute network inputs from them at training time, so every input design is tested on the same recordings. Episodes the expert failed are excluded.

Demonstrations cover only the states the expert visits, so a cloned policy that drifts off them meets states it never saw, and its errors compound [1]. We therefore collect half the data with DART [2]: Gaussian noise is added to the executed forward speed, turning rate, and arm joint references, drawn once per decision and held in between, while the recorded labels stay the expert's noise-free commands. The robot is pushed off the expert's path, and the data records how the expert recovers; wheel noise stops once the base is parked, since the expert no longer corrects its position. Following Laskey et al., the noise matches the learner's own errors: its standard deviations, $0.0156\,\mathrm{m/s}$, $0.137\,\mathrm{rad/s}$, and $0.004\,\mathrm{rad}$, are four times the spread of an early clean-data network's validation errors, widened toward the larger errors of closed loop. Levels re-estimated on the final clean-data network gave the same success (100 and 99 of 100 at $1\,\mathrm{cm}$ and $5\,\mathrm{mm}$), so the result does not hinge on the exact levels. The final dataset has 300 clean and 300 DART episodes with distinct seeds.

C. Inputs and Outputs

Inputs. We give the network exactly the quantities the expert's control laws read, and nothing else (Table I). All are expressed in the base's frame (x ahead, y to the left), so moving and turning the whole scene leaves every input unchanged, as a unit test checks. One rule then holds anywhere on the floor, and 600 episodes need not cover every position and bearing. The last five inputs come from the target's height through the expert's standoff table (Section II-A). They tell the network where the base should stop, which the target's position alone does not; without them, the network could only memorize parking spots (Section IV-A). The heading error enters as its cosine and sine, which avoids the jump at $\pm\pi$.

TABLE I
NETWORK INPUTS, ALL IN THE BASE FRAME, AND THE EXPERT LAW THAT READS EACH ONE

Input	Width	Used by the expert for
Target position	3	heading and reach direction
Hand-to-target error	3	Cartesian arm command
Arm joint angles	7	arm Jacobian and joint limits
Standoff point	2	base law
Distance to standoff circle	1	base law and parked latch
Heading error (cosine, sine)	2	base law and parked latch
Total	18	

Omitted inputs. Each omission has a reason. The robot's own wheel and joint speeds caused causal confusion [3]. Because the expert drives and reaches in separate phases, a moving arm predicts "parked," and a moving base predicts "keep driving." As a result, the network learned to continue whatever the robot was already doing. With speeds, a 10-step network fit the data far better (validation loss 0.024 against 0.082) yet reached no test target in closed loop; without them, it reached 34 of 50. World positions and the bearing would break the invariance above and invite memorization. Elapsed time would offer the same kind of shortcut as speeds, since parking happens at similar times, and the expert uses time only for its timeout. Gravity torques and finger angles carry no information: the fixed $1\,\mathrm{kHz}$ laws compute the first, and the second never changes. The expert's parked latch is not measurable at test time; the parked head infers it (Section II-D).

One instant or an interval. Because the redesigned expert is memoryless, its command depends on the current state alone, so one input step should suffice, and every step of history adds inputs whose noise the network must learn to ignore. The experiments agree. Once speeds were removed, 1- and 10-step histories reached 32 and 34 of 50 targets. On the final recipe and the final seeds, a 10-step MLP reached 99 and 97 targets at $1\,\mathrm{cm}$ and $5\,\mathrm{mm}$ against 100 and 99, with 2.5 times the weights, and it degraded far more under sensor noise (Section IV-E). We therefore use a single step.

Outputs. The network outputs forward speed and turning rate, seven joint reference offsets, seven reference speeds, and the parked probability. Each offset is the expert's joint reference minus the measured angle, which keeps labels small and independent of the arm's pose; the speeds let the $1\,\mathrm{kHz}$ layer advance the reference between decisions, as the expert's does. We clone the expert's behavior, and that behavior lies in its decisions. The expert chooses a forward speed, a turning rate, and a joint reference, and fixed low-level laws, namely the wheel kinematics, the joint PD controller, and gravity compensation from Pinocchio, turn those choices into wheel commands and torques. These laws hold no task knowledge and are identical in every episode, so our policy uses them ex-

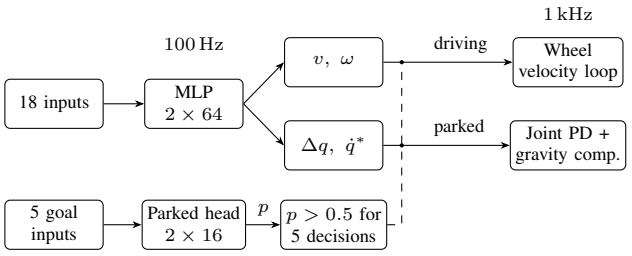


Fig. 2. The learned policy. Every 10 ms the MLP maps 18 inputs to base commands (v, ω) and arm commands (offsets Δq and speeds $\dot{q}^*$), and the parked head's latch selects which are used. Fixed 1 kHz laws, shared with the expert, turn them into wheel speeds and joint torques.

actly as the expert does, and sends the robot wheel commands and torques through the same path. Learning them instead would only add error, and torques held for 10 ms between decisions would lose the PD loop's 1 kHz feedback. Labels are the expert's commands, never the executed ones, so DART noise never enters them.

D. Policy Network and Control

The policy is a multilayer perceptron with two hidden layers of 64 ReLU units, mapping the 18 standardized inputs to the 16 actions (Fig. 2). A separate *parked head*, two hidden layers of 16 units, maps only the five goal inputs to the probability that the base has parked; the network has 6,801 weights in all. The head is separate because a shared one learned a shortcut: since the arm moves only after parking, it judged the base parked whenever the arm moved, and small arm errors in closed loop switched the robot to reaching mid-drive. Reading only the goal inputs, the head can judge parking only from where the base is.

Why two layers of 64? Table II justifies the shape on the development seeds. The expert's laws compose saturations, products such as speed times heading ramp, and a switch at parking, which one hidden layer of 64 fit poorly (82 targets, 9.0 mm). Two layers of 32 failed as well (68), although they used the same 6,801 weights as our final network and instead spent them on a 64-wide head: capacity matters in the body, where the actions are computed. Larger bodies gained nothing in closed loop and lost precision, even where they fit the data better (3×64 reached a validation loss of 0.028 against 0.031), another sign that fitting the recordings more closely does not make a better driver (Section IV-C). Within the head, 16 units matched or beat 8, 32, and 64; across three seeds, 16 averaged 99.3 targets against 97.7 for 64.

Gating. The base counts as parked once the parked probability exceeds one half for five consecutive decisions, which ignores single-decision flickers, and then stays parked, like the expert's latch. Before then, the forward speed and turning rate, capped at the expert's limits, drive the wheels while the arm holds its pose; after, the wheels stop, and the arm's reference becomes the measured angles plus the predicted offsets, advanced at the predicted speeds and clipped as in the expert. The network learns what to do; the fixed laws keep how it is done identical to the expert.

TABLE II

SINGLE-NETWORK SWEEPS ON THE DEVELOPMENT SEEDS, EACH ONE CHANGE FROM THE CHOSEN SETTING (BOLD). SUCCESS AT 1 cm OUT OF 100, AS A MEAN WHERE THREE SEEDS WERE RUN; WEIGHTS INCLUDE A 64-WIDE HEAD UNLESS THE HEAD IS VARIED.

Varied	Setting	Weights	1 cm	Median (mm)
Body	1×64	6,865	82	9.0
	2×32	6,801	68	10.0
	$\mathbf{2 \times 64}$	11,025	**100**	**3.3**
	3×64	15,185	97	4.6
	2×128	25,617	99	4.3
	2×256	79,377	96	4.5
Head	8	6,545	99	4.5
	16 (3 seeds)	6,801	**99.3**	**3.5**
	32	7,697	99	4.3
	64 (3 seeds)	11,025	97.7	3.8
Learning rate	$\mathbf{10^{-3}}$, batch 256		**100**	**3.3**
	10^{-3}, batch 1024		99	4.5
	3×10^{-3}, batch 256		96	5.0
	10^{-2}, batch 256		92	7.4

Ensemble. The final policy averages three networks trained with different seeds, which cancels part of each one's independent error: at 5 mm, the three scored 94, 95, and 99 alone and 99 together. Each runs in plain NumPy with its scaling folded into its weights, avoiding PyTorch's per-call overhead, so the ensemble decides in 38 µs, over 250 times inside the control period.

E. Training

Rows are split 80/20 by whole episode, so validation never shares an episode with training, and we standardize inputs and labels on the training rows. The loss is mean squared error on the standardized actions plus binary cross-entropy between the parked output and the expert's flag; with standardized actions, both terms are of order one, so we sum them unweighted. We use Adam at a learning rate of 10^{-3} and batch size 256, the best of the sweep in Table II, with gradients clipped to norm 1. Training stops after 10 epochs without improvement in validation loss, keeping the best epoch, capped at 100 epochs; doubling the cap left every final network's best epoch unchanged (68, 93, and 90).

Uniform sampling spends most updates on easy rows, while success at a tight tolerance depends on a few rows near the end of the arm's approach. We therefore use prioritized replay [4]. Each row's priority p_i is its latest loss plus a small floor, and each epoch rows are drawn, with repeats, with probability $P_i \propto p_i^{\alpha}$. Importance weights $w_i = (NP_i)^{-\beta}$, scaled so the largest is 1, undo the bias this adds to the gradient. Section IV-C selects $\alpha = 0.3$ with full correction, $\beta = 1$.

F. Compared Models

To test the temporal question directly, we trained an MLP on a 10-step history and an LSTM that reads the same window at each decision. As a non-neural comparison, a mixture

of experts [5] of three XGBoost models [6] resembles the network: a parked classifier on the goal inputs, a base regressor trained before parking, and an arm regressor trained after, with trees of depth 8 and early stopping. A clean-data baseline without replay shows what plain behavior cloning achieves. All share the final model's inputs, split, ensemble size, and controllers.

III. Evaluation Methodology

Every evaluation runs a policy on 100 held-out episodes, none of whose seeds were used to collect training data. We keep three seed sets apart, so that no choice is tuned on the episodes used to report it: development seeds (1000–1099) choose between options, confirmation seeds (5000–5099) check a choice once made, and final seeds (6000–6099) are run once, after every choice was fixed. We report results on the first two as development/confirmation. Because one recipe can vary by several episodes with its training seed alone, we trained the main options with three seeds.

Accuracy. Success is the share of episodes whose hand ends within $1\,\mathrm{cm}$ of the target, the task's own test. Once several policies reached every target, we also report success at a stricter $5\,\mathrm{mm}$, which measures the margin each keeps, and the median final error, which a few failures cannot dominate. Time to park and time to tolerance measure speed.

Efficiency. Model size is the number of learned values: weights for networks, leaves for trees. Decision time is the fastest of five runs of 100 decisions on one CPU thread, through the inference code the robot runs, against the $10\,\mathrm{ms}$ control period.

Ranking. Policies are ranked by success at $1\,\mathrm{cm}$, then at $5\,\mathrm{mm}$, then by median error, then by speed and size. Validation loss is recorded but never used to rank (Section IV-C). When two policies were close, we evaluated both on 500 episodes, with a rule fixed beforehand: a gap of fewer than 8 episodes counts as noise, and the next criterion decides.

Robustness and generalization. On the final seeds, each policy also faces conditions outside its training: *wider ranges* (base starts within $1\,\mathrm{m}$, targets 0.4 to $2.0\,\mathrm{m}$ away and 0.15 to $1.2\,\mathrm{m}$ high); *sensor noise*, Gaussian noise of $\sigma = 2$ or $5\,\mathrm{mm}$ (and mrad) on every measured position and angle the policy sees, while the simulation and success test use the true state; *command noise*, of one or two times the DART levels, on the executed commands; and a *push*, which sets the base sliding sideways at 0.1 or $0.2\,\mathrm{m/s}$, $2\,\mathrm{s}$ into the episode. Each noise draw comes from the episode's seed, so every policy meets the same disturbances.

IV. Results

A. Making Behavior Cloning Work

The first networks fit the template expert's demonstrations closely but failed every closed-loop test episode. Four diagnoses, each tested before the next, turned them into a working policy.

TABLE III
Development of the policy. Each row adds one change to the row above; the final model is in bold. Success out of 100 episodes on the development/confirmation seeds.

Change	1 cm	5 mm	Median (mm)	Weights
Expert	100/100	100/100	1.8	–
Clean data, 2×256	92/97	67/77	5.4	79,377
+ DART data	96/99	77/82	4.5	79,377
Network 2×64	100/100	94/89	3.3	11,025
Parked head $64 \to 16$	100/100	98/94	3.0	6,801
$900 \to 600$ episodes	99/99	87/87	3.4	6,801
Ensemble of 3	100/100	96/97	2.7	3×6,801
+ Prioritized replay	**100/100**	**99/100**	**1.9**	3×6,801

Drift was not the whole story. DART data, the standard remedy for compounding error [2], left success at 0 of 5 test episodes at both noise levels tried.

The parked decision leaked from the arm. Changing one input at a time and watching the outputs showed the network judging the base parked whenever the arm moved, which the separate parked head fixed (Section II-D). Likewise, the template parked on a line from the world origin, so the robot could not tell where to stop, and the network could only memorize parking spots; the goal inputs fixed this (Section II-C).

Two bugs hid outside the loss. Running the expert and the network side by side on identical states showed that the learned policy never loaded the parking goal at reset, and held its arm with different clipping from the expert. Validation loss, computed on recorded rows, could see neither.

The network copied the robot's own motion. With speed inputs, it continued whatever the robot was already doing (Section II-C). Removing them, with the two fixes, gave the first successes (32 and 34 of 50 targets with 1 and 10 steps).

With DART data, this pipeline reached 65 of 100 targets on the realigned template expert, and the redesigned expert raised the same recipe to 96 (Section II-A).

B. Development of the Policy

Table III follows the policy from plain cloning of the redesigned expert to the final model, one change per step. DART data lifted success at $5\,\mathrm{mm}$, and the 2×64 network and 16-wide head (Table II) cut the weights twelvefold while raising it further. Dropping to 600 episodes matched 900 at $1\,\mathrm{cm}$ over three seeds (Table IV) with a third less data, but cost a single network precision at $5\,\mathrm{mm}$; the ensemble recovered most of it, and prioritized replay closed the gap to the expert.

C. Ablations

Data. Table IV varies the final network's training data across three seeds per recipe. Replacing two-thirds of 300 clean episodes with DART episodes raised success at $1\,\mathrm{cm}$ from 78 to 95 with the same amount of data; 300 of each reached 99, and 300 more DART episodes added nothing. The kind of data mattered far more than the amount.

Replay. We swept the priority exponent $\alpha \in \{0.2, 0.3, 0.4, 0.6\}$ with full correction ($\beta = 1$), and

TABLE IV

EFFECT OF TRAINING DATA ON A SINGLE 2×64 NETWORK: SUCCESS AT 1 cm ON THE DEVELOPMENT SEEDS, MEAN OF THREE TRAINING SEEDS (EACH SEED IN BRACKETS).

Training data	Reached at 1 cm
300 clean	78 (79, 89, 67)
100 clean + 200 DART	95 (89, 99, 98)
300 clean + 300 DART	99 (100, 99, 99)
300 clean + 600 DART	99 (100, 100, 98)

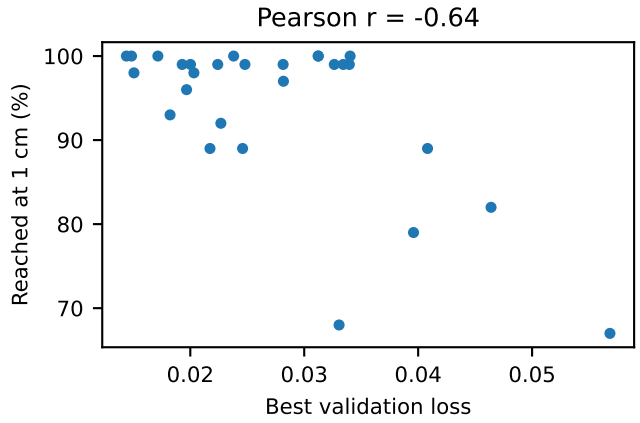


Fig. 3. Best validation loss against success at 1 cm for every sweep network evaluated on its own. Loss tracks success only loosely and cannot separate the good networks.

annealed β from 0.4 to 1 at $\alpha = 0.3$ and at $\alpha = 0.6$, the setting of Schaul et al. At 5 mm, full correction reached 95 to 100 targets, while annealing gave 95/96 at $\alpha = 0.3$ and 97/97 at $\alpha = 0.6$, likely because under-corrected weights let the over-sampled hard rows bias the fit. The two best settings, $\alpha = 0.3$ (99/100) and $\alpha = 0.6$ (100/100), stayed within our tie band over 500 episodes (498 and 500), so median error decided, and we kept $\alpha = 0.3$ (2.2 mm vs. 2.3 mm).

Validation loss. Across the sweep networks, lower validation loss was only loosely associated with success (Pearson $r = -0.64$, Fig. 3), and networks with a loss of about 0.033 ranged from 68 to 100 targets. A learning rate of 3×10^{-3} reached a lower loss than 10^{-3} (0.020 vs. 0.031) but with fewer targets (96 vs. 100), and the training seed alone moved one recipe from 93 to 100. Loss measures fit to recorded states, not driving from the states the policy creates, so we ranked every option in closed loop.

D. Final Results and Model Comparison

Table V gives every model's single run on the final seeds. The final MLP reached every target at 1 cm and 99 at 5 mm, with a 2.3 mm median against the expert's 1.9 mm; plain cloning with the same network reached 92 and 69. It also parked and came within tolerance as quickly as the expert (3.8 s and 6.2 s for both, on the development seeds). Accuracy barely separates the four model families, so efficiency decides.

TABLE V

FINAL SEEDS, RUN ONCE: SUCCESS OUT OF 100, MEDIAN FINAL ERROR, DECISION TIME OF THE ENSEMBLE OF THREE ON ONE CPU THREAD (CONTROL PERIOD 10 000 µs), AND SIZE OF EACH MEMBER.

Model	1 cm	5 mm	Median (mm)	Decision (µs)	Size per model
Expert	100	100	1.9	–	–
MLP (final)	**100**	**99**	**2.3**	**38**	**6,801**
MLP, push training	99	95	3.0	38	6,801
MLP, 10 steps	99	97	2.8	41	17,169
MLP, baseline†	92	69	5.0	38	6,801
LSTM, 10 steps	100	100	2.1	820	56,209
XGBoost	100	100	1.8	13,000	1.4 M*

† Clean data only, no replay. * Leaves, not weights.

TABLE VI

SUCCESS AT 1 cm OUT OF 100 ON THE FINAL SEEDS, OUTSIDE THE TRAINING CONDITIONS. COLUMNS: EXPERT, FINAL MLP, PUSH-TRAINED MLP, 10-STEP MLP, CLEAN-DATA BASELINE, LSTM, XGBOOST.

Condition	Exp.	**Final**	Push	Hist.	Base.	LSTM	XGB
Wider ranges	100	79	76	81	65	80	75
Sensor, 2 mm	100	100	99	96	61	100	100
Sensor, 5 mm	100	79	98	29	5	77	86
Command, 1×	100	100	99	100	78	100	100
Command, 2×	100	99	99	90	38	97	100
Push, 0.1 m/s	100	66	97	53	38	65	99
Push, 0.2 m/s	94	2	57	2	1	2	38

The LSTM matched the MLP within noise: on 500 development episodes under the rule of Section III, the MLP reached 498, and the LSTM 500, and on the final seeds the LSTM's one extra success at 5 mm is again inside that band. It costs 22 times more per decision, has 8 times the weights, and took 12 times longer to train (3196 s against 270 s for three networks). XGBoost was the most precise, at 1.8 mm, but each decision runs three boosted models with about 1.4 million leaves each and takes 13 ms, longer than the control period, so it cannot run in real time; its model files also exceed 100 MB. The final model is the cheapest policy that matches the expert.

E. Generalization and Robustness

Table VI tests every policy on the final seeds under the conditions of Section III. The expert is robust to all but the strongest push, so each drop below it measures the learned policy, not the task.

Command noise is what DART trains for, and the final policy kept 100 and 99 targets at 1× and 2× its levels, versus 78 and 38 for the clean-data baseline; this is DART's clearest effect. *Sensor noise* of 2 mm left it unchanged, but 5 mm, half the success tolerance, cut it to 79. The expert is immune, most likely because its arm integrates its joint reference over time, averaging out the noise; the network instead recomputes its commands from each noisy measurement. The same hidden state that defeats DAgger (Section IV-F) protects the expert here. Ten steps of history made this worse, not better (29), since the network was never trained to average noisy inputs and now meets ten of them. *Wider ranges* cost every learned

policy a fifth to a quarter of its targets: the base-frame inputs make the base's position irrelevant, but heights and distances outside the training ranges remain unseen. *Pushes* were the largest weakness: a sideways slide at $0.1\,\mathrm{m/s}$ leaves the robot in states no demonstration visits, and the final policy reached 66. XGBoost reached 99 without such data, likely because trees hold their nearest learned output outside the data instead of extrapolating, but it cannot run in real time.

Push training. Since the expert's base is memoryless, its labels stay valid wherever a push leaves the robot, unlike DAgger's. We therefore recorded 300 more episodes, each with one sideways push of 0.05 to $0.25\,\mathrm{m/s}$, to either side, 0.5 to $2.5\,\mathrm{s}$ in, drawn from the episode's seed; the expert recovered in 279, which we kept. Three networks trained using all 879 reached 97 targets under $0.1\,\mathrm{m/s}$ pushes and 57 under $0.2\,\mathrm{m/s}$, and also held 98 under $5\,\mathrm{mm}$ sensor noise, so the wider coverage helped beyond pushes. Before testing, we set the rule for replacing the final model: 100/100 at $1\,\mathrm{cm}$ and at least 98/98 at $5\,\mathrm{mm}$ under nominal conditions, plus more successes under pushes. It reached 99/99 and 94/95, so we kept the final model for the task as specified. For deployment, where disturbances and sensing errors are certain, we would choose the push-trained model: among policies that decide within the control period, it leads under pushes and strong sensor noise and remains within one target of the best under every other noise, at a nominal cost of one target at $1\,\mathrm{cm}$, four at $5\,\mathrm{mm}$, and $0.7\,\mathrm{mm}$ of median error.

F. A Negative Result: DAgger

DAgger [1] trains on the states the learner itself reaches: the policy drives while the expert labels every step. In one round, our ensemble drove 300 new episodes, and we retrained three networks on all 900. Success at $5\,\mathrm{mm}$ fell from 96/97 targets to 30/39. A control with the same 300 seeds driven by the expert gave 95/94, so the drop came from the DAgger labels, not the extra data.

The cause is the expert's arm, which integrates a running joint reference rather than computing it from the current state. When the learner drives, that reference follows the motion the expert *would* have made, not the arm's actual motion, so its labels describe a different trajectory. Over rows where the arm reaches, the mean cosine between each row's offset label and its reference-speed label was -0.15 in expert-driven data and $+0.21$ in DAgger data: the relation between the two labels reversed, teaching the network two inconsistent rules. DAgger needs labels that depend on the state alone, which our base already provides and push training exploited (Section IV-E); a memoryless arm would extend this to the whole robot.

V. Conclusion

Behavior cloning of this mobile manipulator worked once the expert was memoryless and the network saw only what the expert's control laws read. On the final seeds, an ensemble of three 6,801-weight MLPs, trained on 600 episodes with DART and prioritized replay, reached all 100 unseen targets at $1\,\mathrm{cm}$ and 99 at $5\,\mathrm{mm}$, against the expert's 100 and 100, in $38\,\mu\mathrm{s}$ per decision. Each choice was justified in closed loop: offline loss could not tell a good driver from a bad one. Both lessons apply to any robot whose expert can be written as feedback on measured state. For deployment, the push-trained variant trades a little nominal precision for far better recovery.

A. Limitations

All training data come from a simulation with perfectly measured state, and the policy weakens under $5\,\mathrm{mm}$ sensor noise, outside the training ranges, and after pushes it was not trained on; real hardware would add delays and unmodeled dynamics that we could not test. The policy inherits the expert's structure, parking before it reaches, and, as a cloned policy, can at best match the expert. Each evaluation has 100 episodes, so differences of a few episodes may not be significant; we confirmed key choices on a second seed set and broke ties on 500 episodes, but most sweeps used a single training seed. Finally, the given arm controller ruled out DAgger (Section IV-F), and thus training in states that the policy reaches but the expert never visits.

B. Improvements

First, a memoryless arm expert that, like our base controller, computes its joint reference from the current state alone would keep its labels valid wherever the learner drives, so that DAgger could add the states the policy actually reaches; filtering the network's inputs, as the expert's integrator does, could reduce the effect of sensor noise. Second, explainability methods such as SHAP and LIME, with trials that occlude one input channel at a time, would measure whether the parked head and the arm rely on the quantities the expert's laws read, and whether the LSTM uses any step but the latest. Third, a world model that predicts several steps ahead, or a policy that predicts chunks of future actions, would let a robot with consecutive targets choose a parking pose that eases the next reach. Finally, on real hardware, the base pose and target would come from localization such as SLAM rather than the simulator; since $5\,\mathrm{mm}$ of sensor noise already cost 21 targets, the policy would need training under the drifting, correlated errors such estimates carry.

References

[1] S. Ross, G. Gordon, and J. A. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proc. 14th Int. Conf. Artificial Intelligence and Statistics (AISTATS)*, ser. PMLR, vol. 15, 2011, pp. 627–635.

[2] M. Laskey, J. Lee, R. Fox, A. Dragan, and K. Goldberg, "DART: Noise injection for robust imitation learning," in *Proc. 1st Conf. Robot Learning (CoRL)*, ser. PMLR, vol. 78, 2017, pp. 143–156.

[3] P. de Haan, D. Jayaraman, and S. Levine, "Causal confusion in imitation learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[4] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, "Prioritized experience replay," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2016.

[5] R. A. Jacobs, M. I. Jordan, S. J. Nowlan, and G. E. Hinton, "Adaptive mixtures of local experts," *Neural Computation*, vol. 3, no. 1, pp. 79–87, 1991.

[6] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.